

Project Overview2

I have slightly updated and adjusted the tasks. The timeline remains unchanged, but there are some modifications to the specific content. Overall, the division of labor has changed very little. These changes are mainly based on the teacher's requirements, and also, to make the specific work more detailed. Everyone needs to choose tasks again (the ones in CA1 were just randomly written). Once we've chosen this time, we'll start working. The overall timeline is quite relaxed; it's roughly three weeks to complete one method, something like that. If you have any questions about the content, methods, interfaces, etc., we can communicate at any time. It seems that the framework might be a bit rigid now, so feel free to speak up in the group if there's anything wrong.

There are 7 people in total: 1 in charge of data, 1 for the system (App), 3 for algorithms, and 2 for visualization.

There are 4 files in total: data_provider.py / algorithms.py / visualizer.py / app.py

File descriptions:

data_provider.py

Goal: Data standardization and discrete slicing

Format unification: Read original CSV or JSON files, map columns with different names to standard formats of start point, end point, and timestamp.

Establish physical ordering: Implement a time-ordered edge list sorted in ascending order of time, which is a prerequisite for all time path algorithms.

Perform snapshot slicing: Split the continuous time flow into snapshot sequences to facilitate frame-by-frame or step-by-step visualization on the front end.

algorithms.py

Goal: Efficient path engine and connectivity theory analysis

Single-pass scanning calculation: Implement the One-pass Temporal BFS algorithm with a time complexity of [missing content], to calculate the earliest arrival time from the source point.

Theoretical support: Integrate calculation logic based on Menger's theorem, quantify the vulnerable points of connectivity by comparing the number of paths before and after edge deletion.

Linear performance guarantee: Ensure that as the number of edges [missing content] increases, the system delay grows linearly rather than exponentially, maintaining stability during large-scale data demonstrations.

visualizer.py

Goal: High frame rate interaction and visual noise reduction

Solve visual congestion: Prevent screen jitter during time evolution and reduce users' cognitive burden through static expansion graphs or fixed node layouts.

High frame rate rendering: Optimize the rendering pipeline to ensure smooth animation switching

and zooming operations when handling hundreds of nodes.

State synchronization drive: Dynamically highlight currently active edges and fade out inactive backgrounds based on the path results returned by the algorithm, achieving a "path-guided" visual effect.

app.py

Goal: System interaction control and performance monitoring

UI construction: Build a web interface based on Streamlit, providing a time slider, edge deletion buttons, and data upload interfaces.

What-if logic scheduling: Capture users' modification instructions (such as delay [missing content]), trigger algorithm recalculation, and notify the visualization module to update the interface.

Engineering quality verification: Perform a stress test of 50 consecutive edge operations, monitor and record system response time, ensuring no memory leaks or rendering freezes.

Task list:

A (app.py): Responsible for build_interface (UI layout) and handle_interactions (state distribution and management).

B (data_provider.py): Responsible for standardize_data (data cleaning and time-based sorting) and slice_snapshots (discrete time slicing).

C (algorithms.py): Responsible for calculate_earliest_arrival (implementation of core temporal breadth-first search) and extract_path_sequence (path backtracking and reconstruction).

D (algorithms.py): Responsible for apply_what_if_modifications (edge/node modification simulation) and evaluate_sensitivity (sensitivity analysis).

E (algorithms.py): Responsible for find_menger_bottlenecks (application of Menger's theorem) and optimize_algorithm_speed (search pruning and efficiency optimization).

F (visualizer.py): Responsible for compute_static_layout (fixed coordinate mapping) and render_snapshot_frame (temporal snapshot rendering).

G (visualizer.py): Responsible for highlight_active_elements (visual noise reduction and highlighting) and compare_scenarios_visual (side-by-side comparison view).

It's normal to be unfamiliar with some technologies or terms. You can also review the CA1 we completed to look at each other's work results.